Perspective-Conditioned Geopolitical Event Prediction

A Full Architecture Specification

Abstract. This document specifies a neural architecture designed to predict whether one real-world entity will take a meaningful action toward another — an invasion, an election, a trade agreement, and so on. The model is unusual in two ways. First, it does not produce a single prediction: it runs N independent sub-models, one for each of N learned *personality perspectives*, and then aggregates their votes. Second, every prediction is conditioned on a set of N global *context vectors* that describe the current state of the world. The final output is a two-number probability — the chance the action happens, and the chance it does not. The exact value of N is left open and will be fixed during experimental design.

Overview and Motivation

Predicting geopolitical events is hard because the same pair of actors behaves differently depending on who is watching and what else is happening in the world. A military analyst, a diplomat, and an economist will weigh the same facts differently. Similarly, a potential invasion looks more or less likely depending on whether there is an ongoing famine, an election cycle, or a diplomatic crisis happening simultaneously.

This architecture encodes both of those ideas directly. It maintains a bank of N *personality vectors* — each one representing a different analytical lens — and a bank of N *context vectors* representing the current global situation. It combines all personalities with all contexts, refines those combinations, and then asks each personality to cast its own probability vote. The votes are pooled into a final answer.

The Four Inputs

Every forward pass receives four things:

- $A \in \mathbb{R}^d$ the **actor** — the entity that might take action (e.g. the US Government, Elon Musk).
- $B \in \mathbb{R}^d$ the **target** — the entity the action would be directed at (e.g. Iran, Twitter/X).
- $\{C_i\}_{i=0}^{N-1},\ C_i \in \mathbb{R}^p$ a bank of N **personality vectors**, each representing a different analytical perspective.
- $\{D_j\}_{j=0}^{N-1},\ D_j \in \mathbb{R}^q$ a bank of N **context vectors**, each encoding a slice of the current global situation.

Three separate embedding spaces

A and B are embedded into a *shared* space. This is important: because they use the same embedding matrix, the geometric distance between A and B carries meaning. Two actors that are ideologically close will end up close in this space; adversaries will be far apart. That distance is exploited directly in Stage 4.

The personality vectors C_i live in their own embedding space, and the context vectors D_j live in yet another. No weights are shared across these three families. This separation ensures that the model cannot confuse "being similar to Iran" with "being a diplomatic personality" — they are literally different coordinate systems.

Stage 1 The Personality–Context Matrix

Why build a matrix?

A personality does not operate in a vacuum — a diplomat's perspective during wartime is different from a diplomat's perspective during peacetime. To capture this, we compute every possible pairing of personality i with context j, producing a matrix $\mathbf{M}$ that holds all N^2 combinations.

Construction

Each cell (i, j) of the matrix is the outer product of personality vector C_i with context vector D_j:

$$\mathbf{M}_{ij} \;=\; C_i \otimes D_j$$

Written out in full, the matrix looks like this:

$$\mathbf{M} = \begin{pmatrix} C_0 \otimes D_0 & C_0 \otimes D_1 & \cdots & C_0 \otimes D_{N-1} \\ C_1 \otimes D_0 & C_1 \otimes D_1 & \cdots & C_1 \otimes D_{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ C_{N-1} \otimes D_0 & C_{N-1} \otimes D_1 & \cdots & C_{N-1} \otimes D_{N-1} \end{pmatrix} \in \mathbb{R}^{N \times N}$$

Reading the matrix row by row: row i contains personality i paired with every possible context. Reading column by column: column j contains context j seen through every possible personality.

Intuition. Think of this matrix as a lookup table. You point to a row ("how would a military analyst see this?") and a column ("given that there is currently a civil war in the region"), and the cell at that intersection holds the combined signal. The model learns what each combination means through training.

Stage 2 Residual Refinement

The problem with raw outer products

The outer product $C_i \otimes D_j$ is a direct, unfiltered combination. It may miss subtle interactions, or amplify noise. We want to give the model the opportunity to *adjust* each cell — sharpening important combinations and suppressing uninformative ones — while always keeping the original signal available. This is what a residual pass does.

One residual pass

A single pass modifies $\mathbf{M}$ as follows:

$$\mathbf{M}^{(t)} = \underbrace{\alpha \cdot \mathbf{M}^{(t-1)}}_{\substack{\text{scaled copy of} \\ \text{previous state}}} + \underbrace{\mathbf{B}}_{\substack{\text{per-cell} \\ \text{learned bias}}} + \underbrace{\mathbf{M}^{(t-1)}}_{\substack{\text{residual:} \\ \text{original preserved}}}$$

Here $\alpha \in \mathbb{R}$ is a single scalar that is shared across every cell and every pass — it controls how much the scaled copy contributes. $\mathbf{B} \in \mathbb{R}^{N \times N}$ is a bias matrix where each cell has its own learned constant, allowing the model to shift individual personality–context combinations up or down independently.

The residual term $(+\,\mathbf{M}^{(t-1)})$ is what makes this a *residual* connection. It guarantees that the original information is never destroyed — the pass can only add to it, never replace it.

Three passes in sequence

We start from $\mathbf{M}^{(0)} = \mathbf{M}$ and run three passes. Expanding them out:

$$\mathbf{M}^{(1)} \;=\; (1+\alpha)\,\mathbf{M}^{(0)} + \mathbf{B}$$

$$\mathbf{M}^{(2)} \;=\; (1+\alpha)\,\mathbf{M}^{(1)} + \mathbf{B}$$

$$\mathbf{M}^{(3)} \;=\; (1+\alpha)\,\mathbf{M}^{(2)} + \mathbf{B} \;\equiv\; \mathbf{M}'$$

After three passes we have the refined matrix $\mathbf{M}'$. Three was chosen as a reasonable starting point that provides enough expressive power without over-processing the signal. It is a hyperparameter that can be tuned.

Intuition. Imagine the raw matrix as a pencil sketch. Each residual pass is like inking over it more carefully — darkening lines that matter, lightening ones that do not — while always keeping the original sketch visible underneath.

Training note. Because α is shared and $\mathbf{B}$ is applied identically in each pass, the three passes are not fully independent transformations. If more expressive refinement is needed, future work could give each pass its own $\alpha^{(t)}$ and $\mathbf{B}^{(t)}$, at the cost of additional parameters.

Stage 3 Projection into Actor–Target Space

At this point $\mathbf{M}'$ lives in personality–context space. Before it can interact with A and B, it must be projected into the actor–target embedding space of dimension d. Two learned weight matrices handle this:

$$\mathbf{Q} \;=\; W_Q\,\mathbf{M}' \;\in\; \mathbb{R}^{d\times N} \qquad\qquad \mathbf{K} \;=\; W_K\,\mathbf{M}' \;\in\; \mathbb{R}^{d\times N}$$

where W_Q, $W_K \in \mathbb{R}^{d\times N}$ are both learned. After this step, the columns of $\mathbf{Q}$ and $\mathbf{K}$ are d-dimensional vectors that speak the same geometric language as A and B.

The naming Q (query) and K (key) is intentional: they follow the attention-mechanism convention where a query vector searches over a set of key vectors. Here the actor–target blend will later act as the query that retrieves relevant personality–context combinations.

Stage 4 Blending the Actor and Target

Measuring alignment with cosine similarity

Before blending A and B, we measure how geometrically similar they are in the shared embedding space:

$$\cos_{AB} \;=\; \frac{A \cdot B}{\|A\|\;\|B\|} \;\in\; [-1,\;1]$$

A value near $+1$ means A and B are pointed in the same direction in embedding space (they share properties, ideology, or behaviour). A value near 0 means they are unrelated. A value near -1 means they are near-opposites. This number becomes the mixing coefficient in the blend that follows.

The blended vector

The core blend is:

$$\boxed{\mathrm{AB}_n \;=\; \tanh\Big(A \;\cdot\; (1-\cos_{AB})\Big) \;+\; \tanh\Big(B \;\cdot\; \cos_{AB}\Big) \;\in\; \mathbb{R}^d}$$

To understand this, look at the two terms separately.

The **left term** $\tanh\big(A(1-\cos_{AB})\big)$ is large when $\cos_{AB}$ is small — that is, when A and B are dissimilar. Intuitively, when the actor and target are very different kinds of entities, the actor's own identity dominates the representation.

The **right term** $\tanh\big(B \cdot \cos_{AB}\big)$ is large when $\cos_{AB}$ is large — when A and B are similar. When the two entities are alike, the target's identity contributes strongly.

Applying tanh to each term before summing serves two purposes: it keeps each component bounded in $(-1, 1)$, and it introduces nonlinearity so the blend is not purely linear in $\cos_{AB}$.

Example. Suppose A is the US and B is Canada (similar, high $\cos_{AB}$). The blend will lean heavily on B: Canada's character shapes the combined representation. Now suppose A is the US and B is North Korea (dissimilar, low $\cos_{AB}$). The blend is dominated by A: the US's own properties drive the combined signal.

Fusing context into the blend

The blend AB_n is further enriched by each context vector D_j through a learned element-wise transform:

$$\mathrm{ABD}_j \;=\; \tanh\Big(w_j \;\odot\; \mathrm{AB}_n \;\odot\; D_j \;+\; b_j\Big) \;\in\; \mathbb{R}^d$$

Here $w_j \in \mathbb{R}^d$ is a learned weight vector and $b_j \in \mathbb{R}^d$ is a learned bias, both unique to context j. The operator $\odot$ denotes element-wise multiplication. The result ABD_j is a vector that carries the actor–target relationship as seen through the lens of global context j.

Stage 5 The N Mini Models

This is the ensemble heart of the architecture. Rather than producing one prediction, the model runs N independent sub-models in parallel, one per personality. Each sub-model: (1) processes its assigned personality vector into a refined form, (2) produces its own two-number probability estimate, and (3) uses that estimate to gate the personality vector before contributing to the final pool.

Step A: normalise the personality vector

Each personality vector C_i is first scaled and normalised:

$$\tilde{C}_i \;=\; \mathrm{softmax}(N \cdot C_i) \;\in\; \mathbb{R}^p$$

Multiplying by N before the softmax sharpens the distribution — it makes the softmax less uniform, so that the most activated dimensions stand out more clearly. After this step, $\tilde{C}_i$ is a proper probability distribution over the p personality dimensions.

Step B: affine transform and ReLU

The normalised personality is projected through a learned affine layer with a ReLU activation:

$$C_i^* \;=\; \mathrm{ReLU}\big(W_i^{(1)}\,\tilde{C}_i \;+\; b_i^{(1)}\big) \;\in\; \mathbb{R}^p$$

Here $W_i^{(1)} \in \mathbb{R}^{p\times p}$ and $b_i^{(1)} \in \mathbb{R}^p$ are weights and biases that are *unique to mini-model i* — no two mini-models share these parameters. The ReLU removes negative activations, producing a sparse, positive-valued representation.

Step C: confidence gating

This is the most distinctive step. The mini-model has already produced a two-number probability estimate $\hat{p}_i = [\hat{p}_i^+, \hat{p}_i^-]$ from the previous forward pass (or from an initialisation). Two copies of C_i^* are made, each scaled by one of the two probability values, and then the tanh of each is summed:

$$C_i'' \;=\; \tanh\!\big(\hat{p}_i^+ \;\cdot\; C_i^*\big) \;+\; \tanh\!\big(\hat{p}_i^- \;\cdot\; C_i^*\big) \;\in\; \mathbb{R}^p$$

The effect is that a mini-model which is very confident (one of $\hat{p}_i^+$ or $\hat{p}_i^-$ close to 1) will push its C_i'' toward the tanh saturation boundary, producing a strong, decisive contribution to the ensemble. A mini-model that is uncertain (both values near 0.5) will produce a softer, more moderate C_i''.

Intuition. Think of this as each personality analyst adjusting the volume of their own voice based on how confident they are. The diplomat who has a strong view shouts; the one who is unsure speaks quietly. The ensemble naturally down-weights uncertain voices without needing an explicit attention mechanism.

Note on circular dependency. At first glance, using $\hat{p}_i$ to gate the same model that produces $\hat{p}_i$ looks circular. It is not: $\hat{p}_i$ is the mini-model's *output*, produced by its own forward pass. The gating happens after that output is computed. During training, gradients flow back through the gating operation into the weights that produce $\hat{p}_i$, which is well-defined.

Building the ensemble matrix

The N gated vectors are collected into a single matrix by stacking them as rows:

$$\mathbf{E} \;=\; \begin{pmatrix} \longleftarrow & C_0'' & \longrightarrow \\ \longleftarrow & C_1'' & \longrightarrow \\ & \vdots & \\ \longleftarrow & C_{N-1}'' & \longrightarrow \end{pmatrix} \;\in\; \mathbb{R}^{N\times p}$$

Row i of $\mathbf{E}$ is the contribution of the i-th personality, already scaled by its own confidence. The matrix $\mathbf{E}$ is the complete ensemble: a structured summary of all N perspectives on the problem.

Stage 6 Polling the Ensemble

Querying all personalities at once

The context-enriched actor–target vector ABD_j is dotted against $\mathbf{E}$ to produce a single N-dimensional score vector:

$$v \;=\; \mathrm{ABD}_j \;\cdot\; \mathbf{E}^\top \;\in\; \mathbb{R}^{1\times N}$$

Each element v_i is the dot product between the current situation (encoded in ABD_j) and the i-th personality's gated view. A high v_i means personality i's perspective is particularly aligned with the current actor–target–context combination. A low v_i means that personality's lens is less relevant here.

> **Intuition.** This is like polling a panel of N analysts. You describe the situation ("Actor A, target B, context D_j") and each analyst responds with a number reflecting how much their expertise applies. The result is a score card across all N analysts.

Stage 7 Bottleneck Refinement

Why refine the poll vector?

The raw poll vector $v \in \mathbb{R}^N$ contains the per-analyst scores, but some of those scores may be redundant or noisy. Three bottleneck passes compress v down to a hidden space of dimension 128 and then back to dimension N. This forces the model to find a compact, information-dense representation of the ensemble opinion before making its final prediction.

One bottleneck pass

$$v^{(t)} \;=\; W_{\downarrow}^{(t)}\,\mathrm{ReLU}\Big(W_{\uparrow}^{(t)}\,v^{(t-1)}\Big)$$

where the two weight matrices handle the expansion and contraction:

$$W_{\uparrow}^{(t)} \in \mathbb{R}^{128\times N} \quad (\text{expands } \mathbb{R}^N \to \mathbb{R}^{128}), \qquad W_{\downarrow}^{(t)} \in \mathbb{R}^{N\times 128} \quad (\text{contracts } \mathbb{R}^{128} \to \mathbb{R}^N)$$

The dimensional journey through one pass:

$$v^{(t-1)} \;\in\; \mathbb{R}^N \;\xrightarrow{W_{\uparrow}}\; \mathbb{R}^{128} \;\xrightarrow{\mathrm{ReLU}}\; \mathbb{R}^{128} \;\xrightarrow{W_{\downarrow}}\; v^{(t)} \;\in\; \mathbb{R}^N$$

This is applied three times, starting from $v^{(0)} = v$, giving $v^{(3)}$ as the final refined poll vector. Each pass has its own independent $W_{\uparrow}^{(t)}$ and $W_{\downarrow}^{(t)}$ — no weights are shared between passes.

Stage 8 Final Output

The refined poll vector $v^{(3)}$ is projected to two dimensions and passed through a softmax to produce a valid probability distribution:

$$\hat{y} \;=\; \mathrm{softmax}\big(W_{\text{out}}\, v^{(3)}\big) \;\in\; \Delta^1$$
$$\hat{y} \;=\; \big[\, \hat{y}^{+},\ \hat{y}^{-} \,\big], \qquad \hat{y}^{+} + \hat{y}^{-} = 1$$

Here $W_{\text{out}} \in \mathbb{R}^{2\times N}$ is a learned output projection. The softmax ensures the two outputs sum to exactly 1, making them proper probabilities.

$\hat{y}^{+}$ is the predicted probability that entity A takes the specified action toward entity B under the given global context. $\hat{y}^{-}$ is the probability it does not.

End-to-End Summary

The full forward pass, written as a chain of transformations:

$$(A,\ B,\ \{C_i\},\ \{D_j\}) \;\longrightarrow\; \mathbf{M} \in \mathbb{R}^{N\times N} \;\xrightarrow{\text{3 residual passes}}\; \mathbf{M}' \in \mathbb{R}^{N\times N}$$

$$\xrightarrow{W_Q, W_K}\; \mathbf{Q},\, \mathbf{K} \in \mathbb{R}^{d\times N} \;\xrightarrow{\cos_{AB}}\; \mathrm{AB}_n \in \mathbb{R}^{d} \;\xrightarrow{\odot D_j}\; \mathrm{ABD}_j \in \mathbb{R}^{d}$$

$$\xrightarrow{N \text{ mini-models}}\; \mathbf{E} \in \mathbb{R}^{N\times p} \;\xrightarrow{\cdot\,\mathbf{E}^{\top}}\; v \in \mathbb{R}^{N} \;\xrightarrow{\text{3 bottleneck passes}}\; v^{(3)} \in \mathbb{R}^{N} \;\xrightarrow{W_{\text{out}}}\; \hat{y} \in \Delta^1$$

Notation Reference

Symbol	Meaning
A, $B \in \mathbb{R}^d$	Actor and target embeddings, shared space
$C_i \in \mathbb{R}^p$	i-th personality vector
$D_j \in \mathbb{R}^q$	j-th context vector
N	Number of personalities / contexts (TBD)
d, p, q	Embedding dimensions
$\mathbf{M}$	$N \times N$ raw outer-product matrix
$\mathbf{M}'$	$\mathbf{M}$ after 3 residual passes
α	Shared scalar residual weight
$\mathbf{B}$	Per-cell bias matrix, $\mathbb{R}^{N \times N}$
W_Q, W_K	Projection matrices into $\mathbb{R}^d$
$\cos_{AB}$	Cosine similarity of A and B
AB_n	Blended actor–target vector
ABD_j	Context-fused actor–target vector
$\hat{p}_i$	2-d probability from mini-model i
C_i^*	Post-ReLU personality representation
C_i''	Confidence-gated personality vector
$\mathbf{E}$	$N \times p$ ensemble matrix
v	Raw poll vector, $\mathbb{R}^N$
$v^{(3)}$	Refined poll vector after 3 bottleneck passes
$\hat{y}$	Final binary probability, $\hat{y} \in \Delta^1$
$\odot$	Element-wise (Hadamard) product
$\otimes$	Outer product

Note: N appears throughout this document as an unspecified integer. It controls the size of the personality bank, the context bank, the cross-product matrix, and the ensemble. It is a fundamental hyperparameter to be fixed during experimental design.